

mongoose_insert_part_1

Dddddd

MongoDB BSON Data Types				
Data Type	BSON Type Number	Size	Use	
String	2	Variable	Name, Email, Address	
Int32	16	4 bytes	Age, Marks	
Int64 (Long)	18	8 bytes	Large IDs, Big Numbers	
Double	1	8 bytes	Decimal approximation	
Decimal128	19	16 bytes	Money, Salary	
Boolean	8	1 byte	true / false	
Date	9	8 bytes	DOB, CreatedAt	
Timestamp	17	8 bytes	Replication, Internal	
ObjectId	7	 12 bytes	Default <code>_id</code>	

mongoose_insert_part_1

ObjectId	7	12 bytes	Default <code>_id</code>	 
Array	4	Variable	Skills, Subjects	
Object (Document)	3	Variable	Nested Documents	
Null	10	-	Null values	
Binary	5	Variable	Files, Images	
Regular Expression	11	Variable	Regex	
JavaScript	13	Variable	JS Code (Rare)	
MinKey	-1	-	Internal	
MaxKey	127	-	Internal	

SQL vs MongoDB

SQL

MongoDB

INT

Int32

BIGINT

Int64 (Long)

FLOAT

Double

DECIMAL

Decimal128

VARCHAR

String

CHAR

String

TEXT

String

BOOLEAN

Boolean

DATE



▼	BOOLEAN	Boolean
	DATE	Date
	TIMESTAMP	Timestamp / Date
	BLOB	Binary
	Table	Collection
	Row	Document
	Column	Field

MongoDB BSON Data Types

[Upgr](#)

Data Type	BSON Type Number	Size	Use	📄
String	2	Variable	Name, Email, Address	
Int32	16	4 bytes	Age, Marks	
Int64 (Long)	18	8 bytes	Large IDs, Big Numbers	
Double	1	8 bytes	Decimal approximation	
Decimal128	19	16 bytes	Money, Salary	
Boolean	8	1 byte	true / false	
Date	9	8 bytes	DOB, CreatedAt	
Timestamp	17	8 bytes	Replication, Internal	
ObjectId	7	12 bytes	Default <code>_id</code>	

ObjectId	7	12 bytes	Default <code>_id</code>	 
Array	4	Variable	Skills, Subjects	
Object (Document)	3	Variable	Nested Documents	
Null	10	-	Null values	
Binary	5	Variable	Files, Images	
Regular Expression	11	Variable	Regex	
JavaScript	13	Variable	JS Code (Rare)	
MinKey	-1	-	Internal	
MaxKey	127	-	Internal	

```

1  const mongoose = require("mongoose");
2  const userSchema = new mongoose.Schema({
3
4      phone: {
5          type: String,
6          required: true,
7          unique: true,
8          match: /^[789][0-9]{9}$/
9      },
10
11     password: {
12         type: String,
13         required: true,
14         minlength: 16,
15         maxlength: 16,
16         match: /^(?=[A-Z])(?=[a-z])(?=[A-Z])(?=[0-9])(?=[@#$%])[A-Za-z0-9@#%]{16}$/
17     },
18
19     dob: {
20         type: String,
21         required: true,
22         match: /^(0[1-9]|12)[0-9]|3[01])\/(0[1-9]|1[0-2])\/[0-9]{4}$/
23     },
24 });
25
26
27 const User = mongoose.model("User", userSchema);

```

</> JavaScript

```
async function main() {  
  
    await mongoose.connect(  
        "mongodb://localhost:27017/test"  
    );  
  
    try {  
  
        const user = await User.create({  
  
            phone: "9876543210",  
  
            password: "Abcdefgh12345@#$",  
  
            dob: "18/08/2002"  
  
        });  
  
        console.log(user);  
  
    }  
    catch (err) {
```



</> JavaScript

```
const users = await User.insertMany([

  {
    phone: "9876543210",
    password: "Abcdefgh12345@#$",
    dob: "18/08/2002"
  },

  {
    phone: "8765432109",
    password: "Xabcdef1234567@$",
    dob: "25/12/2001"
  },

  {
    phone: "7654321098",
    password: "Pqwerty1234567@#",
    dob: "05/01/2003"
  }

]);

console.log(users);
```



```
> JS query.js > [e] userSchema > city > enum
1  const userSchema = new mongoose.Schema({
2    username: {
3      type: String,
4      required: true,
5      minlength: 3,
6      maxlength: 30,
7      trim: true,
8      match: /^[A-Za-z0-9_]+$/,
9      unique: true,
10     index: true
11   },
12
13   email: {
14     type: String,
15     required: true,
16     trim: true,
17     lowercase: true,
18     unique: true
19   },
20 }
```

```

    },
    password: {
      type: String,
      required: true,
      minlength: 16,
      maxlength: 16,
      match: /^(?=[A-Z])(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@#$])[A-Za-z0-9@#${16}$/
    },
    city: {
      type: String,
      default: "Indore",
      enum: ["Indore", "Bhopal", "Ujjain", "Dewas"]
    }
  });

```

VALIDATION

- ├ required
- ├ minlength
- ├ maxlength
- ├ enum
- ├ match
- └ validate

TRANSFORMATION

- ├ trim
- ├ lowercase
- └ uppercase

DATABASE / INDEX

- ├ unique
- └ index

DEFAULT

- └ default

★ Upgra

Ye distinction interview me "Mongoose schema ke String type ke options kaun-kaun se hain?" wale question me kaafi useful hai.



</> JavaScript

```
const userSchema = new mongoose.Schema({  
  
  name: {  
    type: String,  
    required: true,  
    minlength: 3,  
    maxlength: 50,  
    trim: true,  
    lowercase: true,  
    uppercase: false,  
    enum: ["Rahul", "Amit", "Tarun"],  
    match: /^[A-Za-z ]+$/,  
    unique: true,  
    index: true,  
    default: "Unknown"  
  }  
  
});
```



// JavaScript

```
db.users.insertMany([
  {
    _id: ObjectId(),
    name: "Tarun Patidar",           // string
    age: 22,                         // number
    salary: 45500.75,                // double
    isActive: true,                  // boolean
    phone: "9876543210",             // phone number
    aadhaar: "123456789012",         // aadhaar number
    password: "Tarun@1234Abcd$X",    // password
    dob: new Date("2004-08-15"),     // date
    middleName: null,                // null
    tempField: undefined,            // undefined

    address: {                       // embedded document
      city: "Indore",
      state: "Madhya Pradesh",
      pincode: 452010
    },

    skills: ["C++", "MongoDB", "Node.js"], // array
```

```
education: [                                     // array of embedded documents
  { level: "10th", year: 2018, percentage: 82.4 },
  { level: "12th", year: 2020, percentage: 78.6 }
],

extraInfo: { hobby: "Cricket" },                // schemaless field
profilePic: BinData(0, "U29tZURhdGE="),          // binary data
createdAt: new Date()
},

{
  _id: ObjectId(),
  name: "Priya Verma",
  age: 24,
  salary: 52000.50,
  isActive: false,
  phone: "8123456789",
  aadhaar: "987654321098",
  password: "Priya#5678Xyzm@Q",
  dob: new Date("2002-03-20"),
  middleName: null,
```



```
// javascript
middleName: null,

address: {
  city: "Bhopal",
  state: "Madhya Pradesh",
  pincode: 462001
},

skills: ["Java", "SQL"],
education: [
  { level: "BTech", college: "LNCT", cgpa: 8.1 }
],

extraInfo: { favoriteColor: "Blue", expectedCTC: 600000 },
createdAt: new Date()
}
])
```

Default behavior

```
</> JavaScript  
  
{ ordered: true }
```

Meaning

- Documents **sequence me insert** honge
- Agar beech me **error** aa gaya, to **uske baad wale documents** insert nahi honge

Example

```
</> JavaScript  
  
db.users.insertMany(  
  [  
    { _id: 1, name: "Tarun" },  
    { _id: 1, name: "Priya" }, // duplicate _id -> error  
    { _id: 3, name: "Rahul" }  
  ],  
  { ordered: true }  
)
```



2) Unordered Insert

Syntax

```
</> JavaScript  
  
{ ordered: false }
```

Meaning

- Agar kisi ek document me error aaye, tab bhi **baaki valid documents insert hote rahenge**

Example

```
</> JavaScript  
  
db.users.insertMany(  
  [  
    { _id: 1, name: "Tarun" },  
    { _id: 1, name: "Priya" }, // duplicate _id -> error  
    { _id: 3, name: "Rahul" }  
  ],  
  { ordered: false }
```

